

Database Ingestion & Edge Collector Architecture

Overview

The Predictive Maintenance System (PMS) ingests telemetry from two distinct server roles:

1. **Database server** — traditional pull-based feeds (job history, query store, wait stats, IO stats, etc.) collected by the scheduler via database connectors.
2. **Application server** — push-based feeds sent by an **Edge Collector** agent running on the app server.

Both feed types produce `ParsedLogEntry[]` and flow through the same ingestion pipeline (`POST /api/ingest/batch`).

Phase 1-6: Database-Server Feeds (Pull)

FeedId	Source	Connector
jobHistory	SQL Agent job history	MSSQL / Postgres / MySQL
queryStore	Query performance stats	MSSQL / Postgres / MySQL
spExec	Stored procedure execution stats	MSSQL / Postgres / MySQL
waitStats	Wait statistics	MSSQL / Postgres / MySQL
ioStats	IO latency & throughput	MSSQL / Postgres / MySQL
xevents	Extended Events / pg_stat_activity	MSSQL / Postgres / MySQL
errorLog	Database error log	MSSQL / Postgres / MySQL
osMetrics	OS-level metrics (CPU, RAM, disk) from DB host	MSSQL / Postgres / MySQL

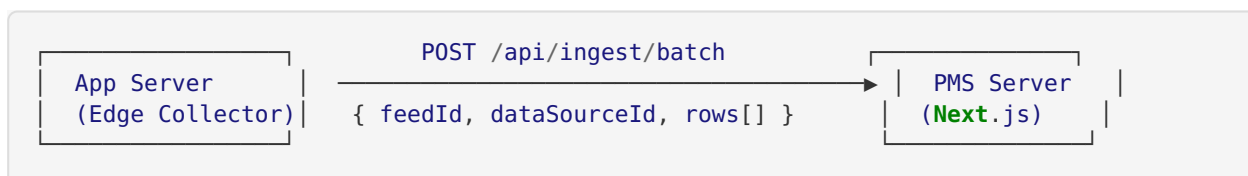
The scheduler (`lib/scheduler/dispatch.ts`) pulls these feeds on a configurable interval.

Phase 7: Application-Server Feeds (Push via Edge Collector)

These feeds cover the **split-server gap** — the application tier and the network between app and DB.

FeedId	Parsed Source	What it captures
appMetrics	App.Metrics	Response time p95/p99, error rate, thread pool usage, GC time, queue length
appLogs	App.Logs	Structured application logs (Serilog/NLog/OTel), pattern-based severity classification
windowsEventLog	Windows.EventLog	Windows Application/System event log, knows critical EventIDs (.NET crash, service terminated, app pool failure)
iisLogs	IIS.AccessLog	IIS W3C access logs — HTTP status, latency, queue length, error rate summary
networkMetrics	Network.Metrics	App↔DB network: RTT latency, TCP retransmissions, connection pool usage, ephemeral port exhaustion, DNS resolution, packet loss

Edge Collector Pattern



- The Edge Collector is a lightweight agent deployed on the application server.
- It pushes telemetry to PMS via `POST /api/ingest/batch` with `{ feedId: FeedId, dataSourceId: string, rows: RawRow[] }`.
- No changes needed to the batch endpoint — the new feeds flow through the existing pipeline.
- The `DataSource` model is the registration point (represents the monitored system).

Cross-Server Correlation

- `Application.hostname` = app server, `DataSource.host` = DB server
- `DataSource.applicationId` FK connects them
- The EAI engine sees both sources under the same Application and correlates timestamps
- **NTP time sync is critical** for cross-server correlation

EAI Categories (Phase 7)

Category	Pattern Match	Diagnostic Focus
<code>app_server_pressure</code>	<code>/app\.(metrics logs)/i</code>	App-tier resource exhaustion, thread pool starvation
<code>network_degradation</code>	<code>/network\.?metrics/i</code>	Network latency, retransmissions, connection pool exhaustion
<code>windows_event_failure</code>	<code>/windows\.?event/i</code>	.NET crashes, service failures, app pool recycling
<code>iis_error_spike</code>	<code>/iis\.(access log)/i</code>	HTTP 5xx spikes, request queue overflow

Mapper Files

All edge mappers live in `lib/mappers/edge/` and are registered in `lib/mappers/edge/index.ts`. They are pure functions (no IO, no Prisma) — same pattern as the DB mappers.

Key Design Principle

“Performance degradation often isn’t because the code is slow or the DB is slow — it’s because the conversation between them is lagging or failing.”

The `networkMetrics` feed specifically addresses this blind spot. You need all three signal types:

- **Logs** → explain failures
- **Metrics** → show trends
- **Traces** → show causality